

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет прикладної математики та інформатики
Кафедра дискретного аналізу та інтелектуальних систем

КУРСОВА РОБОТА

**РОЗРОБКА ІНТЕЛЕКТУАЛЬНОЇ СИСТЕМИ АВТОМАТИЧНОГО
РОЗПІЗНАВАННЯ ТА ВІДСТЕЖЕННЯ НОМЕРНИХ ЗНАКІВ
ТРАНСПОРТНИХ ЗАСОБІВ У ВІДЕОПОТОЦІ НА ОСНОВІ ГІБРИДНИХ
МЕТОДІВ КОМП'ЮТЕРНОГО ЗОРУ**

ЗМІСТ

ВСТУП.....	4
РОЗДІЛ 1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ІСНУЮЧИХ МЕТОДІВ	6
1.1 Постановка задачі та технічні вимоги до системи	6
1.1.1 Математична постановка задачі	6
1.1.2 Вимоги до точності та продуктивності	6
1.1.3 Вплив умов використання на стабільність роботи системи	7
1.2 Аналіз існуючих систем розпізнавання номерних знаків	8
1.3 Аналіз структури та стандартів номерних знаків України	9
1.3.1 Геометричні обмеження та фільтрація	9
1.3.2 Синтаксичний шаблон та позиційна валідація	9
1.3.3 Обмеження алфавіту та обґрунтування правил корекції.....	10
1.4 Математичні методи попередньої обробки зображень та детекції меж	11
1.4.1 Фільтрація шумів та розмиття за Гауссом	11
1.4.2 Адаптивне вирівнювання гістограми (CLAHN)	11
1.4.3 Детектор країв Кенні	11
1.4.4 Автоматична бінаризація за методом Оцу	12
1.5 Аналіз методів локалізації та розпізнавання символів.....	12
1.5.1 Перспективне вирівнювання.....	12
1.5.2 Архітектура EasyOCR та модель CRNN	13
1.6 Алгоритми відстеження об'єктів та стабілізації результатів	13
1.6.1 Відстеження за центроїдами	13
1.6.2 Статистична стабілізація результатів	14
1.6.3 Механізм очищення пам'яті	14
РОЗДІЛ 2. ПРОЄКТУВАННЯ ТА АЛГОРИТМІЧНІ РІШЕННЯ СИСТЕМИ.....	15
2.1 Архітектура програмного комплексу та оркестрація модулів	15
2.1.1 Модульна структура та компоненти системи	15
2.1.2 Організація багатопотокової обробки даних	17
2.2 Проєктування модуля детекції номерних знаків	18
2.2.1 Алгоритм попередньої обробки зображення	18
2.2.2 Геометрична селекція та апроксимація об'єктів	19
2.3 Проєктування модуля розпізнавання тексту.....	19
2.3.1 Алгоритми попередньої обробки зон інтересу	19
2.3.2 Нейромережеве розпізнавання символів.....	21

2.4 Модуль трекінгу та часової стабілізації	21
2.4.1 Алгоритм відстеження об'єктів	22
2.4.2 Метод мажоритарного голосування (Rolling Vote)	22
2.5 Модуль інтелектуальної валідації та верифікації результатів	23
РОЗДІЛ 3. ПРОГРАМНА РЕАЛІЗАЦІЯ ТА АНАЛІЗ РЕЗУЛЬТАТІВ ДОСЛІДЖЕННЯ	24
3.1 Обґрунтування вибору програмних засобів реалізації.....	24
3.2 Опис графічного інтерфейсу користувача	25
3.3 Аналіз результатів експериментальних досліджень.....	27
3.3.1 Оцінка точності детекції та розпізнавання символів.....	27
3.3.2 Дослідження швидкодії та продуктивності системи	30
3.3.3. Порівняння з існуючими аналогами.....	30
3.4 Тестування модулів програмного комплексу	32
ВИСНОВКИ	33
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	34

ВСТУП

Актуальність теми. У сучасному світі технології розпізнавання номерних знаків на основі штучного інтелекту стали не лише одним із напрямів застосування, а й необхідністю. Зі стрімким збільшенням кількості автомобілів постала критична необхідність в ефективному моніторингу трафіку, управлінні паркувальними майданчиками та забезпеченні безпеки на дорозі. Традиційні методи ідентифікації транспортних засобів людиною-оператором є малоефективними через високу швидкість потоку та людський фактор. Використання інтелектуальних систем розпізнавання номерних знаків дозволяє автоматизувати ці процеси. Розпізнавання номерних знаків є одним із ключових завдань комп'ютерного зору, яке є складним через велику кількість варіацій умов зйомки відеоматеріалу: атмосферні явища (сніг, дощ), розмиття внаслідок руху, низька якість відео та зміна ракурсу зйомки (нахили та кути). Розробка гібридних методів, що поєднують класичний комп'ютерний зір та глибоке навчання, є актуальним завданням для підвищення точності та стабільності ідентифікації об'єктів.

Мета дослідження — розробити та дослідити інтелектуальну систему автоматичного розпізнавання та відстеження номерних знаків транспортних засобів у відеопотоці на основі гібридних методів комп'ютерного зору, що забезпечує стійкість до дестабілізуючих чинників реального середовища.

Для досягнення поставленої мети необхідно виконати наступні завдання:

- проаналізувати сучасні методи попередньої обробки зображень для покращення контрастності та читабельності тексту в складних умовах;
- реалізувати алгоритм детекції об'єктів на основі контурного аналізу та геометричних перетворень для корекції перспективи;
- інтегрувати нейромережеву модель оптичного розпізнавання символів (OCR) для стабільної ідентифікації знаків;
- розробити алгоритм відстеження (tracking) об'єктів та модуль інтелектуальної постобробки результатів на основі накопиченої статистики.

Об'єкт дослідження – процеси цифрової обробки відеоданих та автоматичної ідентифікації текстової інформації на рухомих об'єктах у відеопотоці.

Предмет дослідження – алгоритми та програмні засоби для детекції контурів, розпізнавання символів за допомогою нейронних мереж та методи асоціації даних для відстеження декількох транспортних засобів одночасно.

Методи дослідження. Для розв'язання поставлених завдань обрано мову програмування Python, яка є стандартом у сфері аналізу даних. Основу практичної реалізації становлять методи комп'ютерного зору, впроваджені за допомогою бібліотеки OpenCV; зокрема, використано контурний аналіз для детекції номерних знаків та афінні перетворення для корекції перспективи зображення. Процес розпізнавання символів базується на використанні архітектури глибокого навчання, реалізованої в бібліотеці EasyOCR. Для забезпечення стабільної ідентифікації кількох об'єктів одночасно застосовано апарат прикладної математики: обчислення Евклідової відстані між центроїдами об'єктів дозволяє реалізувати алгоритм відстеження (tracking) та асоціювати дані у динамічному відеопотоці. Також використано методи статистичного аналізу для накопичення історії розпізнавання та вибору найбільш імовірного значення номера.

РОЗДІЛ 1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ІСНУЮЧИХ МЕТОДІВ

У першому розділі проведено детальний огляд сучасного технологічного розпізнавання номерних знаків, проаналізовано технічні вимоги до систем відеоаналітики та розглянуто основні перешкоди, що виникають під час обробки зображень у реальному часі.

1.1 Постановка задачі та технічні вимоги до системи

Автоматичне розпізнавання номерних знаків (Automatic License Plate Recognition, ALPR) безпосередньо є однією з найбільш затребуваних і водночас складних підзадач комп'ютерного зору. Математична суть задачі зводиться до вилучення структурованої текстової інформації з відеоматеріалу високої розмірності.

1.1.1 Математична постановка задачі

Нехай вхідними даними є часова послідовність зображень (кадрів) $V = \{I_1, I_2, \dots, I_n\}$, де кожен кадр I_t представляє собою матрицю пікселів розмірністю $W \times H$. Задача системи розпізнавання полягає у знаходженні відображення $F: I_t \rightarrow \{S, ID, T\}$, де:

- S — розпізнані символи реєстраційного номера;
- ID — унікальний ідентифікатор транспортного засобу, присвоєний алгоритмом трекінгу;
- T — час фіксації об'єкта.

Процес реалізації функції F у цій роботі полягає у наступних завданнях: відшукуванні об'єкта та його локалізації, нормалізації геометричних спотворень, класифікації завдяки нейромережі та корекції значень.

1.1.2 Вимоги до точності та продуктивності

Для систем інтелектуального моніторингу трафіку встановлюються певні вимоги щодо точності та продуктивності:

- ймовірність правильності ідентифікації. Для практичного застосування показник точності повинен бути не меншим за 95%. У цій роботі особливу увагу приділено зниженню кількості помилок типу «False Recognition», за яких схожі символи розпізнаються неправильно.
- режим реального часу (Real-time processing). Система повинна обробляти вхідний потік зі швидкістю не менше 25 кадрів за секунду (FPS). Це досягається шляхом використання апаратного прискорення CUDA на графічних процесорах архітектури NVIDIA.
- стійкість до динамічних шумів. Алгоритми повинні ігнорувати об'єкти, що не є номерними знаками (наприклад рекламні вивіски, написи на бортах вантажівок, тощо).

1.1.3 Вплив умов використання на стабільність роботи системи

На відміну від ідеальних тестових варіантів, у реальних умовах дані містять похибки.

Основні подразники, що перешкоджають розпізнаванню є:

- освітленість: прямі сонячні промені, відблиски від фар у нічний час або тіні на білих автомобілях створюють проблеми низького контрасту між фоном номера та символами.
- погодні умови: атмосферні опади (дощ, сніг) створюють додатковий візуальний шум, який може перекривати частину знака або спотворювати його контури.
- динаміка об'єкта: ефект розмиття, що виникає при русі автомобіля на високій швидкості компенсується методами накопичення історії кадрів.

Використання гібридного підходу, який поєднує класичні фільтри OpenCV та сучасні OCR-моделі на базі глибокого навчання (EasyOCR), дозволяє мінімізувати вплив цих факторів за рахунок багаторівневої обробки кожного кадру.

1.2 Аналіз існуючих систем розпізнавання номерних знаків

Перед проєктуванням власного рішення необхідно розглянути існуючі системи ALPR, визначити їх можливості та обмеження.

Одним із таких є Platerrecognizer (Parkrow Inc.) [9], хмарний комерційний сервіс із REST API, що підтримує номерні знаки понад 100 країн та забезпечує високу точність завдяки спеціалізованим нейромережевим моделям. Сервіс має зручну інтеграцію через API. Незважаючи на переваги, сервіс має також певні недоліки, такі як:

- повна залежність від інтернет-з'єднання;
- платна модель (безкоштовний тариф обмежений кількістю запитів на місяць);
- закритий вихідний код, що унеможливорює адаптації під специфічні умови без придбання комерційної ліцензії.

Наступним є OpenALPR [7] — бібліотека з відкритим вихідним кодом, що реалізує підхід на основі класичного комп'ютерного зору (без глибокого навчання). Перевагою цієї системи є можливість повністю локального розгортання та безкоштовне використання. Система має наступні недоліки:

- нижча точність у складних умовах освітлення порівняно з нейромережевими рішеннями,
- відсутність механізму відстеження об'єктів між кадрами,
- обмежена підтримка нестандартних форматів.

Останнім розглянутим варіантом у курсовій роботі є OpenCV [8] + Tesseract [13] (типове відкрите рішення) — це поширений підхід у наукових роботах та прототипах, що базується на детекції через контурний аналіз OpenCV із подальшим розпізнаванням символів через Tesseract OCR. Цей підхід є повністю безкоштовним та локальним. Недоліки у цьому підході:

- Tesseract не оптимізований для номерних знаків, а тому точність на реальних відео суттєво нижча, ніж у спеціалізованих рішеннях;
- відсутній трекінг об'єктів.

Підсумовуючи, комерційні рішення забезпечують вищу точність, але є закритими та платними. Відкриті рішення доступні, але мають обмеження у точності та функціональності. Це зумовлює доцільність розробки системи, яка поєднує переваги обох підходів: локальну роботу на основі відкритих інструментів із нейромережовим розпізнаванням через EasyOCR[5] та механізмом стабілізації результатів.

1.3 Аналіз структури та стандартів номерних знаків України

Ефективність розпізнавання безпосередньо залежить від формату номерів, яку можна представити. У межах цієї курсової роботи об'єктом дослідження обрано реєстраційні знаки типу 1-1 (згідно з ДСТУ 4278:2019 [1]), що є найбільш розповсюдженим стандартом для легкових автомобілів в Україні.

1.3.1 Геометричні обмеження та фільтрація

Для локалізації об'єкта основним чинником є геометрична форма, а саме прямокутник. Стандартний номерний знак має фіксоване співвідношення сторін приблизно 4.5:1. У системі локалізація базується на контурному аналізі, який полягає у пошуку чотирикутників достатнього розміру. Також враховується контрастність символів, адже чорні символи на білій рамці створюють градієнт яскравості, що застосовується для роботи детектора Кенні. Це дозволяє системі виявляти номерні знаки навіть при недостатньому освітленні.

1.3.2 Синтаксичний шаблон та позиційна валідація

Український стандарт номерних знаків передбачає структуру із 8 символів, яку можна подати у вигляді шаблону LL NNNN LL, де перші дві літери – регіональний префікс, що вказує на місце реєстрації автомобіля; позиції 3-6 містять чотири цифри від 0 до 9, а позиції 7 та 8 позначають серію номерного знака.

Така структура дозволяє прописати алгоритм інтелектуальної корекції та постобробки. Через те, що нейронні мережі часто можуть припуститися помилки

під час розпізнавання схожих символів, знання структури номерів дозволяють виправляти ці помилки.

1.3.3 Обмеження алфавіту та обґрунтування правил корекції

Однією з важливих особливостей державного стандарту є обмеженість набору використовуваних символів. Спочатку стандарт базувався на використанні 12 літер кириличного алфавіту, які мають аналог у графічному накреслені латинського алфавіту: А, В, С, Е, Н, І, К, М, О, Р, Т, Х.

Проте впровадження нових цифрових сервісів реєстрації транспортних засобів (зокрема через «Дію» та електронний кабінет водія) у 2023–2025 роках призвело до появи нових буквених серій, таких як DI, PD, ED, EE. Оскільки ці символи (зокрема D) не мають кириличних аналогів, але входять до стандартного латинського набору знаків, у цій роботі прийнято рішення використовувати повний латинський алфавіт (A–Z) як основу для розпізнавання. Це забезпечує інваріантність системи до майбутніх змін у законодавстві.

Існування синтаксичного шаблону дозволяє реалізувати етап подальшої корекції (OCR post-processing). Імовірність помилки зростає за умов низької роздільної здатності або поганого ракурсу зйомки. Часті джерела помилок є візуально схожі знаки, такі як:

- цифра 0 та літери O, D;
- цифра 1 та літери I, J;
- цифра 8 та літери В;
- цифра 2 та літера Z;
- цифра 5 та літера S.

У програмному забезпеченні є алгоритм валідації, який базується на аналізі позицій. Наприклад, якщо на позиціях 3–6 виявлено символ, що належить до групи літер, він автоматично трансформується у відповідну цифру за метрикою візуальної схожості.

1.4 Математичні методи попередньої обробки зображень та детекції меж

Якість розпізнавання тексту в потоковому відео суттєво залежить від етапу попередньої обробки (preprocessing), метою якого є мінімізація цифрового шуму та посилення контрастності корисного сигналу.

1.4.1 Фільтрація шумів та розмиття за Гауссом

Відеопотік часто містить шуми. Теоретичною основою згладжування є фільтр Гаусса, математично описаний формулою [4]:

$$G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}},$$

де σ – середньоквадратичне відхилення, що визначає ступінь розмиття. У практичній реалізації системи замість стандартного фільтра Гаусса застосовано двостороннє (білатеральне) згладжування `cv2.bilateralFilter`, яке, на відміну від Гауссового, зберігає гострі межі об'єктів, згладжуючи лише однорідні ділянки фону.

1.4.2 Адаптивне вирівнювання гістограми (CLAHE)

Для роботи програми в умовах нерівномірного освітлення використано алгоритм CLAHE [8] (Contrast Limited Adaptive Histogram Equalization). На відміну від класичного методу глобального вирівнювання гістограми, який обробляє все зображення як єдиний масив даних, CLAHE розбиває зображення на невеликі фрагменти і обробляє їх локально.

Для обмеження посилення шуму в однорідних областях алгоритм застосовує функцію відсікання, що дозволяє перерозподілити яскравість таким чином, щоб зберегти деталізацію символів на фоні номерної пластини.

1.4.3 Детектор країв Кенні

Для знаходження меж використовується багатоетапний алгоритм Кенні [2, 8]. Його математична основа базується на пошуку екстремумів градієнта яскравості зображення та включає наступні етапи:

- обчислення векторів градієнта G_x, G_y за допомогою оператора Собеля;
- визначення амплітуди градієнта за формулою $G = \sqrt{G_x^2 + G_y^2}$;
- аналіз напрямку градієнта $\theta = \arctan\left(\frac{G_x}{G_y}\right)$;
- пригнічення немаксимумів для отримання тонких контурних ліній.

Це дозволяє системі виділити чіткий контур прямокутної форми номерного знака серед інших об'єктів у кадрі.

1.4.4 Автоматична бінаризація за методом Оцу

Для сегментації символів від фону використано поріг Оцу. Математична суть методу полягає у виборі такого порогу T , який мінімізує внутрішньокласову дисперсію $\sigma_W^2(T)$ [2], що визначається як зважена сума дисперсій двох класів (фону та об'єкта):

$$\sigma_W^2(T) = w_0(T)\sigma_0^2(T) + w_1(T)\sigma_1^2(T)$$

Це дозволяє системі автоматично адаптуватися до яскравості кадру, що робить алгоритм розпізнавання універсальним для денної та нічної зйомки.

1.5 Аналіз методів локалізації та розпізнавання символів

Коли у програмі вже виконана попередня обробка зображень, то наступним етапом є детекція та видалення текстової інформації. У цій роботі використано комбінований підхід, який в свою чергу поєднує геометричний аналіз та нейромережеві обчислення.

1.5.1 Перспективне вирівнювання

Зазвичай під час зйомки є кут розташування камери до площини номерного знака. Через це виникають перспективні викривлення, що призводять до зниження точності розпізнавання. Тому використано метод гомографії для вирівнювання області інтересу. Розв'язується система лінійних рівнянь, яка прив'язує чотири знайдені контури з координатами ідеального прямокутника. З системи знаходиться

матриця M розмірністю 3×3 , результатом якої стає фронтальна проекція номера, придатна для якісного розпізнавання символів.

1.5.2 Архітектура EasyOCR та модель CRNN

Для розпізнавання символів обрано бібліотеку EasyOCR [5], що базується на архітектурі CRNN [3] (Convolutional Recurrent Neural Network). Модель поєднує:

- згорткові шари (CNN), які виділяють візуальні ознаки символів (лінії, кути, тощо);
- рекурентні шари (LSTM), які аналізують послідовність символів для кращого розпізнавання знаків на основі контексту;
- шар CTC (Connectionist Temporal Classification), який декодує вихідну послідовність у фінальну текстову стрічку.

1.6 Алгоритми відстеження об'єктів та стабілізації результатів

Завершальним етапом обробки є відстеження рухомого об'єкта у часі. Потрібно накопичувати статистичні дані для кожного транспортного засобу, оскільки на кадрах можливий варіант, коли номера частково перекриті.

1.6.1 Відстеження за центроїдами

Щоб розрізняти автомобілі у кадрі використовується алгоритм відстеження за центроїдами. За допомогою моментів зображення обчислюються координати геометричного центру (c_x, c_y) для кожного контуру.

$$c_x = \frac{M_{10}}{M_{00}} \quad c_y = \frac{M_{01}}{M_{00}}$$

Для ідентифікації об'єкта в новому кадрі використовується обчислення Евклідової відстані d між старими центроїдами та новими.

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Якщо відстань d не перевищує встановлений поріг (у цій роботі – 120 пікселів), тоді об'єкт вважається незмінним і йому присвоюється унікальний ідентифікатор (ID).

1.6.2 Статистична стабілізація результатів

Розпізнавання тексту на різних кадрах однієї і тієї ж машини може відрізнятися через динамічні перешкоди (наприклад, «BC1234AA» на одному кадрі та «B01234AA» на іншому). Щоб цьому завадити, в системі використано метод накопичення історії розпізнавання. Кожна машина має своє ID, для якого створюється черга результатів. Із цієї черги за допомогою статистичного аналізу вибирається мода послідовності. Це дозволяє зменшити випадкові помилки нейромережі та знаходити стабільний результат.

1.6.3 Механізм очищення пам'яті

Кожен об'єкт має свій «час життя». Це впроваджено для забезпечення тривалої роботи системи в реальному часі. Кожному ідентифікатору присвоюється лічильник кадрів, який оновлюється при кожному успішному виявленні. Якщо протягом 25 послідовних ітерацій об'єкт не з'явився у кадрі, то його дані видаляються з оперативної пам'яті. Реалізовано це, щоб запобігти неконтрольному зростанню обсягу даних та забезпечити стабільну швидкість обробки.

РОЗДІЛ 2. ПРОЄКТУВАННЯ ТА АЛГОРИТМІЧНІ РІШЕННЯ СИСТЕМИ

У другому розділі розглядається практична реалізація системи. Основну увагу приділено опису архітектурних рішень, опису логіки взаємодії певних модулів та обґрунтуванню вибору алгоритмів реалізації.

Ціль розділу – це деталізація процесу перетворення вхідного відеосигналу на текстову інформацію. Для досягнення високої точності та продуктивності системи було розроблено модульну структуру, що поєднує методи комп'ютерного зору, неймережеві технології розпізнавання символів та алгоритми інтелектуальної корекції помилок на основі державних стандартів.

2.1 Архітектура програмного комплексу та оркестрація модулів

Програма написана з урахуванням об'єктно-орієнтованого підходу, щоб забезпечити модульність, збільшити швидкість обробки даних та зручно тестувати окремі компоненти. Головною архітектурною ідеєю є відокремлення обчислювальної логіки («ядра») від графічного інтерфейсу та механізмів збору даних.

2.1.1 Модульна структура та компоненти системи

У системі кожен етап обробки даних інкапсульовано в межах окремого програмного пакету.

Проектна ієрархія системи включає наступні ключові блоки:

- обчислювальне ядро (core), яке містить базові класи для локалізації об'єктів (PlateDetector), інтелектуального розпізнавання тексту (PlateRecognizer), відстеження центроїдів (CentroidTracker) та логічної валідації результатів (PlateValidator).
- рівень управління конвеєром (pipeline), який представлений класом-оркестратором ALPRPipeline, що координує послідовність викликів модулів ядра та агрегує проміжні результати обробки кожного кадру.

- інтерфейсний шар (ui), який забезпечує візуалізацію результатів, керування джерелами відеосигналу та відображення бази розпізнаних номерів у реальному часі.
- конфігураційний модуль (config), який централізовано зберігає параметри алгоритмів, що дозволяє гнучко налаштовувати систему під різні умови освітлення та формати номерних знаків без зміни вихідного коду.

Для наочного представлення взаємодії цих компонентів у межах єдиного циклу обробки розроблено структурну схему конвеєра (рис. 2.1).

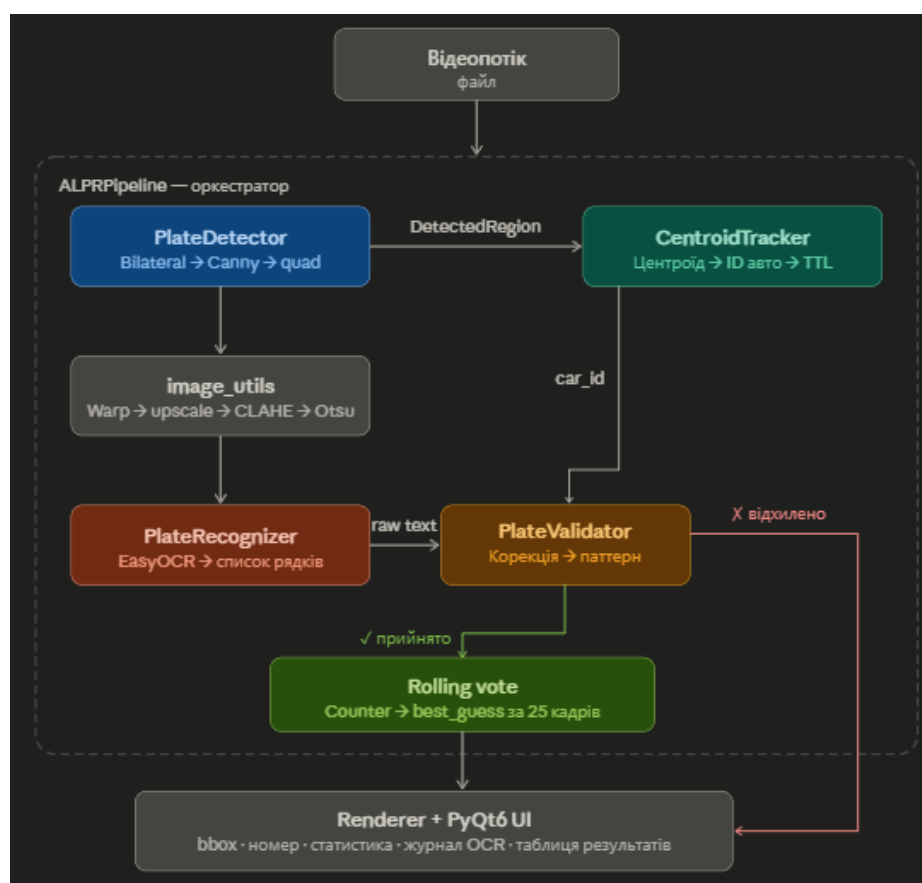


Рисунок 2.1 Архітектурна схема конвеєра обробки даних (Pipeline) системи ALPR.

Як продемонстровано на рис. 2.1, конвеєр забезпечує наскрізну передачу даних: від виділення зони інтересу в модулі PlateDetector до фінального статистичного вибору номера в блоці Rolling vote. Важливою особливістю архітектури є те, що ідентифікатор автомобіля (car_id), згенерований трекером, супроводжує об'єкт на всіх етапах розпізнавання та валідації. Це дозволяє системі не просто виводити миттєві результати з окремих кадрів, а накопичувати історію

розпізнавань для кожного конкретного транспортного засобу, суттєво підвищуючи загальну точність системи за рахунок вибору найбільш частотного результату (моди) з накопиченої послідовності.

Для деталізації об'єктно-орієнтованої структури обчислювального ядра системи, демонстрації взаємодії внутрішніх полів, атрибутів конфігурації та методів ключових класів розроблено UML-діаграму класів програмного комплексу, яку представлено на рисунку 2.2.

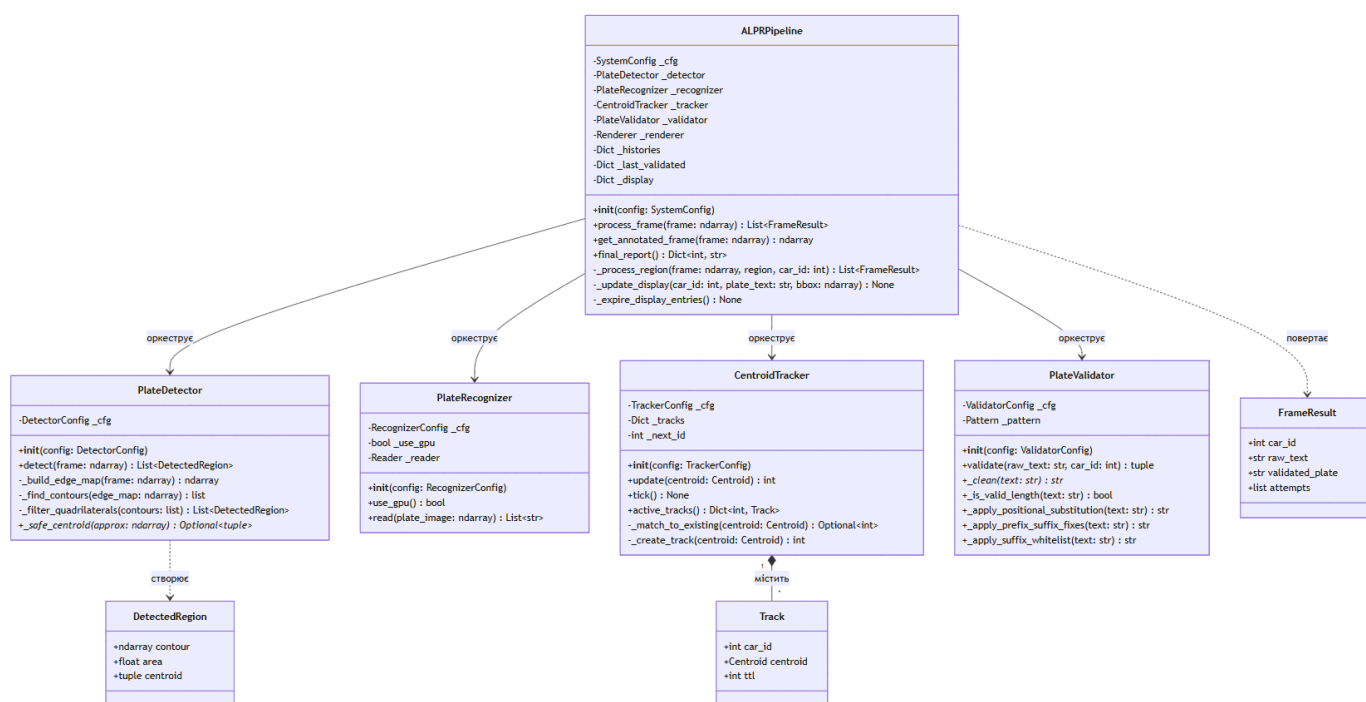


Рисунок 2.2. UML-діаграма класів обчислювального ядра системи ALPR.

2.1.2 Організація багатопотокової обробки даних

Задачі OCR характеризуються своєю високою обчислювальною складністю. Якщо запустити послідовне виконання, це призведе до значних затримок у виконанні та створенні графічного інтерфейсу. Для уникнення зупинок, у програмі прописана багатопотокова архітектура на базі фреймворку PyQt6 [10].

Основна логіка розподілу задач покладена на клас VideoWorker, який функціонує в окремому фоновому потоці (QThread). Це дозволяє ізолювати два критичні процеси:

- фоновий потік обробки (Worker Thread), який здійснює зчитування кадрів із джерела відео, запускає обчислювальний конвеєр ALPR Pipeline та видає кадр із результатами;
- головний потік інтерфейсу (Main GUI Thread), який відповідає виключно за відображення вікна програми, обробку подій користувача та оновлення таблиці розпізнаних номерів.

Використовується механізм сигналів та слотів для безперешкодної взаємодії між потоками. Коли кадр оброблений, VideoWorker відправляє сигнал в головний потік для миттєвого відображення результатів. Завдяки цьому підтримується максимальна частота кадрів (FPS), не зважаючи на час який нейромережа витрачає на обробку кадрів.

2.2 Проектування модуля детекції номерних знаків

За перший етап обробки – виявлення номерного знаку (ROI) на повнорозмірному кадрі - відповідає модуль локалізації PlateDetector. Він нівелює проблеми, які виникають через різні умови освітлення та кут нахилу камери.

2.2.1 Алгоритм попередньої обробки зображення

Для забезпечення стабільної роботи детектора в умовах змінного освітлення, вхідний кадр проходить обробку в класі PlateDetector через метод `_build_edge_map`. Технічна реалізація включає наступні кроки:

- адаптивна фільтрація шуму. Замість стандартного розмиття за Гауссом використано функцію `cv2.bilateralFilter` [8] з параметрами ($d=11$, $\sigma_{\text{Color}}=17$, $\sigma_{\text{Space}}=17$). Це дозволяє згладити дрібні шуми на фоні (асфальт, рослинність), зберігаючи при цьому гостроту градієнтів на межах номерної пластини.
- обчислення карти градієнтів. Детекція меж реалізована за допомогою алгоритму Кенні. Вибір порогів ($\text{low}=30$, $\text{high}=200$) зумовлений необхідністю отримання замкнених контурів навіть при низькому контрасті номера відносно кузова автомобіля.

Реалізація у коді методу `_build_edge_map` (фрагмент):

Error! Bookmark not defined.2.2.2 Геометрична селекція та апроксимація об'єктів

Подальшим кроком є аналіз топології знайдених ліній. Для відсікання нерелевантних об'єктів у методі detect реалізована багаторівнева фільтрація:

- апроксимація полігонів: для кожного знайденого контуру викликається функція cv2.approxPolyDP. Параметр точності є встановлено як фіксоване значення 10 пікселів, що дозволяє системі ідентифікувати чотирикутники, навіть якщо їхні краї мають незначні дефекти або закруглення.
- фільтрація за площею та пропорціями: система ігнорує контури, площа яких менша за 800 пікселів (далекі об'єкти).
- визначення опорних точок: для кожного валідного об'єкта обчислюється центроїд, який стає вхідними даними для наступного модуля трекінгу.

В результаті цих фільтрацій створюється масив координат потенційних номерних знаків. Таким чином обчислювальні витрати, які були б використані на етапі розпізнавання тексту, зменшуються завдяки обробці лише цільових зон.

2.3 Проектування модуля розпізнавання тексту

Модуль розпізнавання (PlateRecognizer) поєднує методи препроцесингу зображень для усунення спотворень та інтелектуальний аналіз тексту на основі глибокого навчання. Основною метою цього модуля є отримання максимально точного символічного рядка з урахуванням специфіки шрифтів та можливих дефектів пластини номера.

2.3.1 Алгоритми попередньої обробки зон інтересу

Оскільки локалізовані ділянки кадру часто мають низьку роздільну здатність або перспективні викривлення, перед розпізнаванням застосовується ланцюжок

трансформацій (рис. 2.3). Це критично важливо для стабільної роботи неймережевих моделей, які чутливі до нахилу та контрастності вхідних даних.

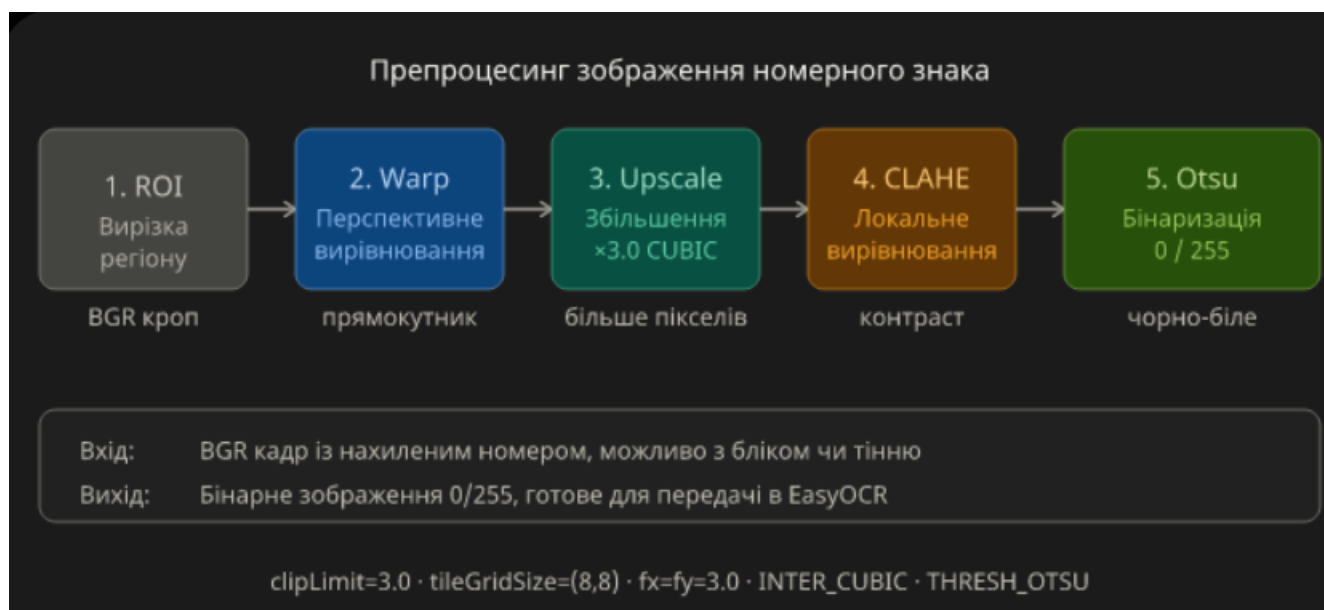


Рисунок 2.3 Етапи попередньої обробки локалізованого зображення.

Технічна реалізація препроцесингу в модулі `image_utils` включає:

- Perspective Warp: усунення нахилу номера шляхом трансформації знайденого чотирикутника у правильний прямокутник. Це нівелює спотворення, що виникають через ракурс зйомки камери.
- Upscale (x3.0): збільшення зображення за допомогою кубічної інтерполяції (`cv2.INTER_CUBIC`), що дозволяє OCR-моделі краще розрізняти межі символів на невеликих за розміром фрагментах.
- CLAHE: адаптивне вирівнювання гістограми з обмеженням контрасту (`clipLimit=3.0`), що забезпечує чітке виділення символів навіть за наявності тіней або нерівномірного освітлення.

2.3.2 Нейромережеве розпізнавання символів

Для безпосереднього зчитування тексту обрано бібліотеку EasyOCR, яка використовує комбіновану архітектуру глибокого навчання. Процес аналізу поділяється на два паралельні етапи:

- детекція тексту (модель CRAFT): нейронна мережа визначає конкретні області всередині ROI, де знаходяться символи, ігноруючи рамки номера або прапори.
- розпізнавання (CNN + LSTM): згорткова мережа вилучає ознаки літер, а рекурентна мережа аналізує їх як послідовність, враховуючи контекст сусідніх знаків.

Модуль повертає не лише розпізнаний рядок, а й коефіцієнт впевненості (confidence score), що дозволяє відсіяти результати з високим рівнем шуму ще до етапу валідації.

Реалізація препроцесингу зони інтересу (фрагмент):

```
roi = cv2.resize(roi, None, fx = 3.0, fy = 3.0,
                 interpolation = cv2.INTER_CUBIC)
clahe = cv2.createCLAHE(clipLimit = 3.0, tileGridSize = (8, 8))
gray = clahe.apply(cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY))
_, binary = cv2.threshold(gray, 0, 255,
                          cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```

2.4 Модуль трекінгу та часової стабілізації

Для стабільності системи під час обробки відео, реалізовано механізм часової стабілізації. Завдяки цьому, вплив короткочасних спотворень кадру значною мірою зменшується. Під словами спотвореннями кадру можна навести такі приклади: відблиски, рух камери чи часткове закриття номерного знака, які можуть призводити до помилок розпізнавання.

2.4.1 Алгоритм відстеження об'єктів

У системі реалізовано модуль CentroidTracker, який базується на евклідовій відстані між центроїдами об'єктів. Кожному новому детектовному об'єкту присвоюється унікальний ідентифікатор `car_id`, який зберігається за ним протягом усього часу перебування в кадрі.

Процес трекінгу включає такі етапи:

- обчислення центроїдів для всіх нових чотирикутників, знайдених детектором;
- розрахунок матриці відстаней між новими центроїдами та центроїдами об'єктів з попереднього кадру;
- асоціація об'єктів з мінімальною відстанню та оновлення їхніх координат;
- видалення об'єктів, які не з'являлися в кадрі протягом заданої кількості кадрів (параметр `max_disappeared`).

Реалізація алгоритму асоціації центроїдів (фрагмент):

```
distances = dist.cdist(existing_centroids, [centroid])
nearest_idx = int(distances.argmin())
if distances[nearest_idx][0] <
    self._cfg.max_centroid_distance:
    return track_ids[nearest_idx]
```

2.4.2 Метод мажоритарного голосування (Rolling Vote)

Для фінального результату використовується метод статистичного аналізу отриманих результатів, записаних для кожного `car_id`. Алгоритм роботи Rolling vote полягає у наступному:

- для кожного автомобіля зберігається історія розпізнавань за останні 25 кадрів;
- для кожного кадру в буфері проводиться валідація отриманого тексту;
- після накопичення достатньої кількості даних виконується пошук моди у списку розпізнаних номерів.

2.5 Модуль інтелектуальної валідації та верифікації результатів

Останнім етапом конвеєра є проходження отриманими результатами процедури верифікації на відповідність державним стандартам. Метою роботи модуля PlateValidator є приведення результатів оптичного розпізнавання до уніфікованого вигляду та перевірка їхньої логічної цілісності. На відміну від етапу OCR, де відбувається аналіз візуальних ознак, даний модуль оперує символьними послідовностями та правилами їхньої побудови.

Реалізація позиційної корекції символів у PlateValidator (фрагмент):

```
for i in letter_positions:
    chars[i] = digit_to_letter_map.get(chars[i], chars[i])
for i in digit_positions:
    chars[i] = letter_to_digit_map.get(chars[i], chars[i])
```

Процес обробки результату в межах модуля реалізовано як послідовність таких перевірок:

- нормалізація та позиційний аналіз. Виконується очищення рядка від шумів та зіставлення його структури із регулярними виразами, що описують державні стандарти України. Це дозволяє відсіяти результати, що за кількістю символів або їхнім типом не можуть належати до номерних знаків.
- контекстна корекція символів. На основі знання про структуру номера проводиться заміна візуально схожих символів (наприклад, цифра «0» замінюється на літеру «О» у літерних секціях і навпаки). Цей крок критично важливий для усунення типових помилок нейромережі при роботі з подібними зображеннями номерних знаків.
- верифікація за переліком дозволених ідентифікаторів. Фінальний результат зіставляється із базою даних санкціонованих об'єктів («білим списком») шляхом точного порівняння суфіксів із переліком дозволених регіональних кодів.

РОЗДІЛ 3. ПРОГРАМНА РЕАЛІЗАЦІЯ ТА АНАЛІЗ РЕЗУЛЬТАТІВ ДОСЛІДЖЕННЯ

У цьому розділі описано програмні засоби, що були використані під час розробки системи, розглянуто архітектуру графічного інтерфейсу користувача, а також наведено результати експериментів на різних наборах даних.

3.1 Обґрунтування вибору програмних засобів реалізації

У даній системі розпізнавання номерних знаків було обрано такий стек, який створює умови для ефективного поєднання алгоритмів комп'ютерного зору із різними нейромережевими моделями. Вибором мови програмування став Python [12], адже ця мова містить спеціалізовані бібліотеки та має високу швидкість розробки програмних рішень для наукових досліджень. За середовище написання було обрано Visual Studio Code, що підтримує інтеграцію із віртуальними середовищами та засобами модульного тестування.

У ході виконання програми було використано такі бібліотеки та фреймворки:

- бібліотека OpenCV [8] (Open Source Computer Vision Library), що застосовується для виявлення номерів та попередньої обробки зображення. Вона забезпечує виконання низькорівневих операцій, таких як двобічна фільтрація, детекція меж за методом Кенні та апроксимація контурів;
- фреймворк EasyOCR [5], який обраний як базовий засіб для оптичного розпізнавання символів. Його використання дозволяє інтегрувати готові нейромережеві моделі архітектури CNN+LSTM, що демонструють високу точність на зображеннях із низькою роздільною здатністю;
- бібліотека PyQt6 [10], яка була використана для розробки графічного інтерфейсу та організації багатопотокової архітектури системи. Завдяки цьому відокремлюється високозатратний обчислювальний процес від потоку візуалізації;

- бібліотека NumPy [6], що застосовується для ефективної роботи з формулами при виконанні матричних операцій та обчисленні центроїдів.

Застосування такого стеку технологій дозволило реалізувати гнучку модульну систему, яка відповідає принципам простоти та стислості викладу. Отримано можливість для проведення повторних числових експериментів та подальшого масштабування системи.

3.2 Опис графічного інтерфейсу користувача

Графічну частину програми реалізовано на базі фреймворку PyQt6. Головним завданням інтерфейсу є відображення процесу розпізнавання у реальному часі та представлення користувачу засобів для моніторингу перебігу процесу.

Візуальна частина вікна програми розроблена так, щоб забезпечити зручне та логічне розташування основних інформаційних зон, інтуїтивно зрозумілих для користувача. Центральну частину вікна займає область відтворення потоку, де відображається вихідний сигнал із накладеними графічними примітивами (рамка для локалізації та текст розпізнаного номерного знаку). Знизу розташовано панель керування, що містить набір для запуску та зупинки процесу обробки, а також вибору джерела вхідних даних. Із правої сторони розміщено таблицю реєстрації подій, таких як динамічний список результатів чи логування перебігу валідації даних, а згори розміщений блок статусів, де відображаються такі показники, як продуктивність FPS, кількість авто та кількість розпізнаних номерів.

Загальний вигляд розробленого інтерфейсу користувача під час роботи системи наведено на рисунках 3.1 та 3.2

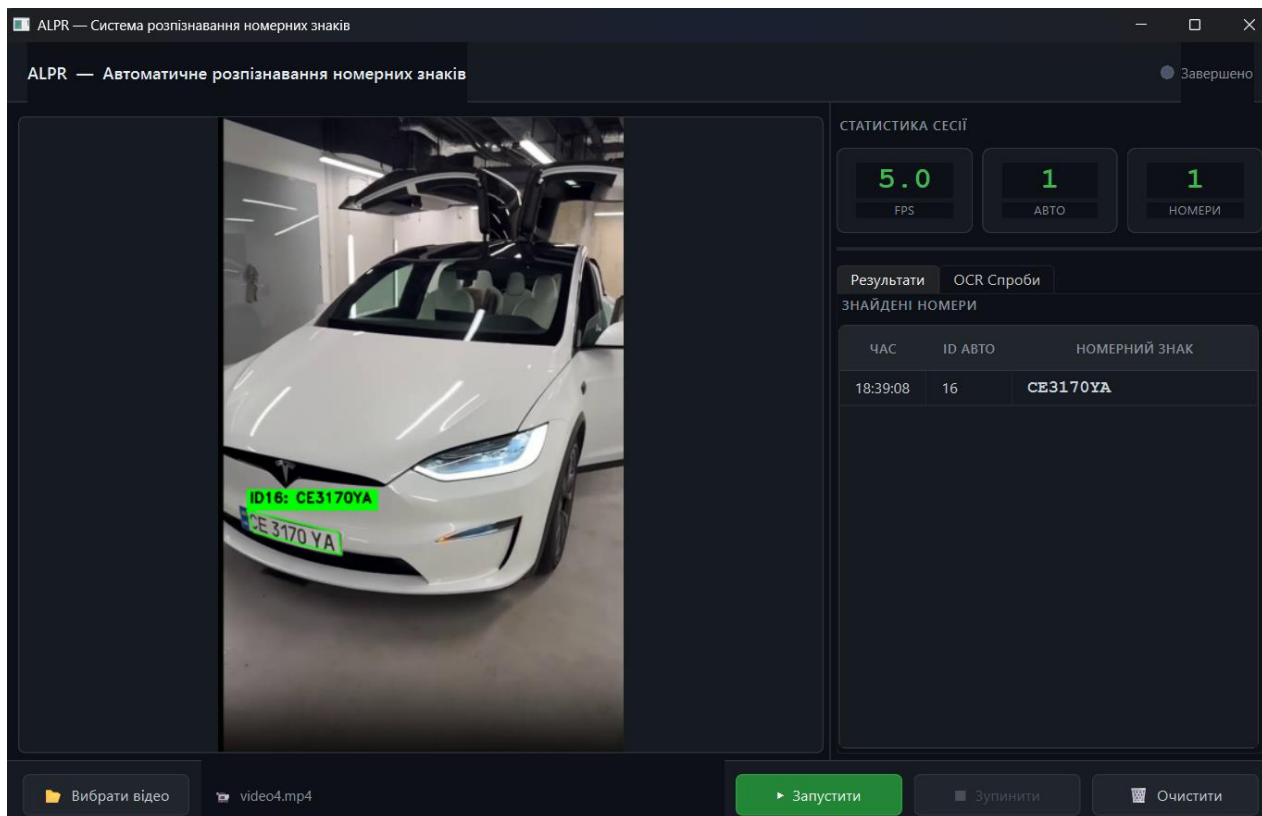


Рисунок 3.1 Основне вікно програми із результатами детекції та статистикою сесії.

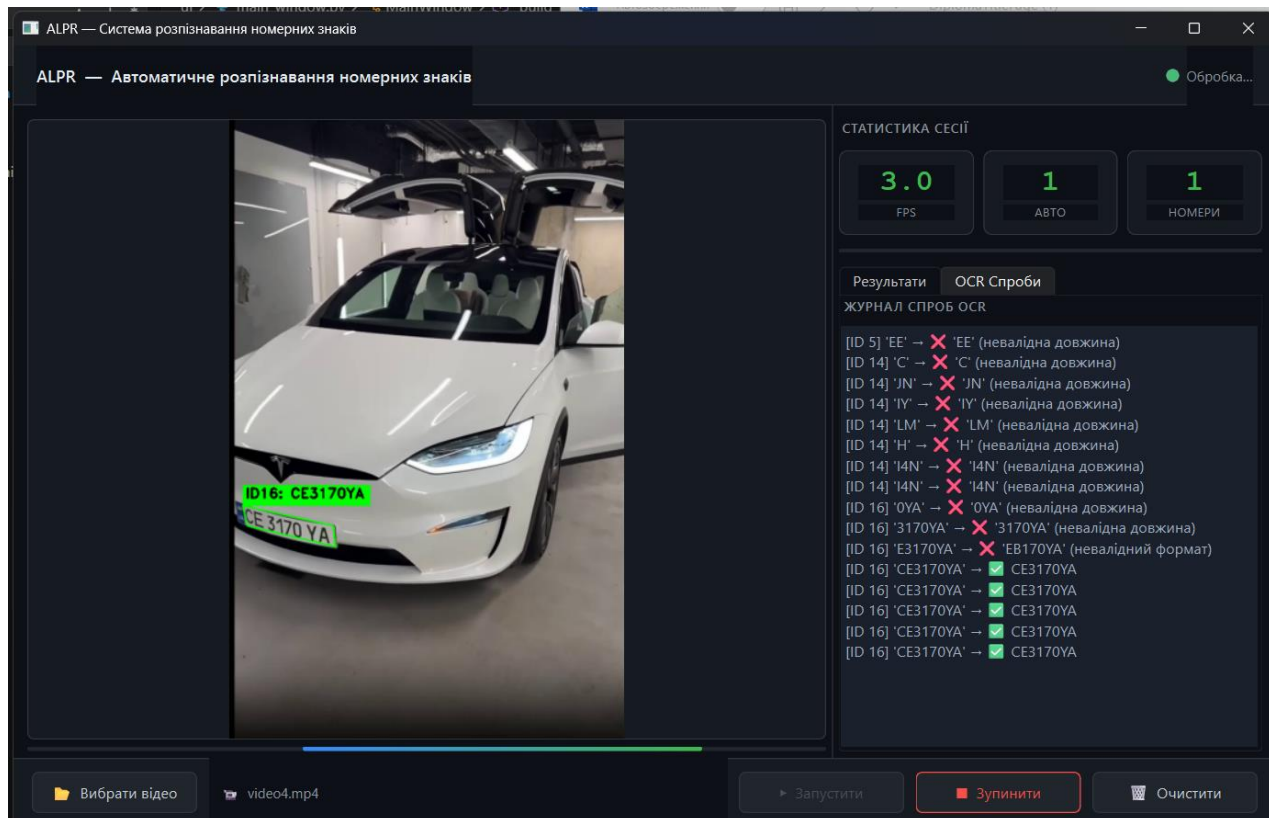


Рисунок 3.2 Журнал спроб розпізнавання (OCR) із відображенням процесу валідації символів.

На рисунку 3.2 продемонстровано роботу модуля валідації, який у реальному часі фільтрує розпізнані послідовності за критеріями довжини та формату, що відповідає алгоритмам, описаним у підрозділі 2.5

3.3 Аналіз результатів експериментальних досліджень

Щоб зможти підтвердити ефективність запропонованих алгоритмів, було проведено серію числових експериментів. Перевірка роботи системи здійснювалася на основі реальних відеоданих, зафіксованих у складних погодних умовах та при різному ступені динаміки руху транспортних засобів. А критеріями оцінки виступили точність ідентифікації номерного знаку та стабільність відстеження об'єктів у відеопотоці.

3.3.1 Оцінка точності детекції та розпізнавання символів

Експериментальна оцінка точності системи базувалася на аналізі журнальних записів (OCR логів) та візуальному контролю результатів детекції. Замість усереднених статистичних показників, ефективність інструментарію була підтверджена через дослідження роботи алгоритмів у різних сценаріях.

На рисунку 3.3 представлено результат роботи системи в умовах складних погодних умов (засніжене покриття, низький контраст).

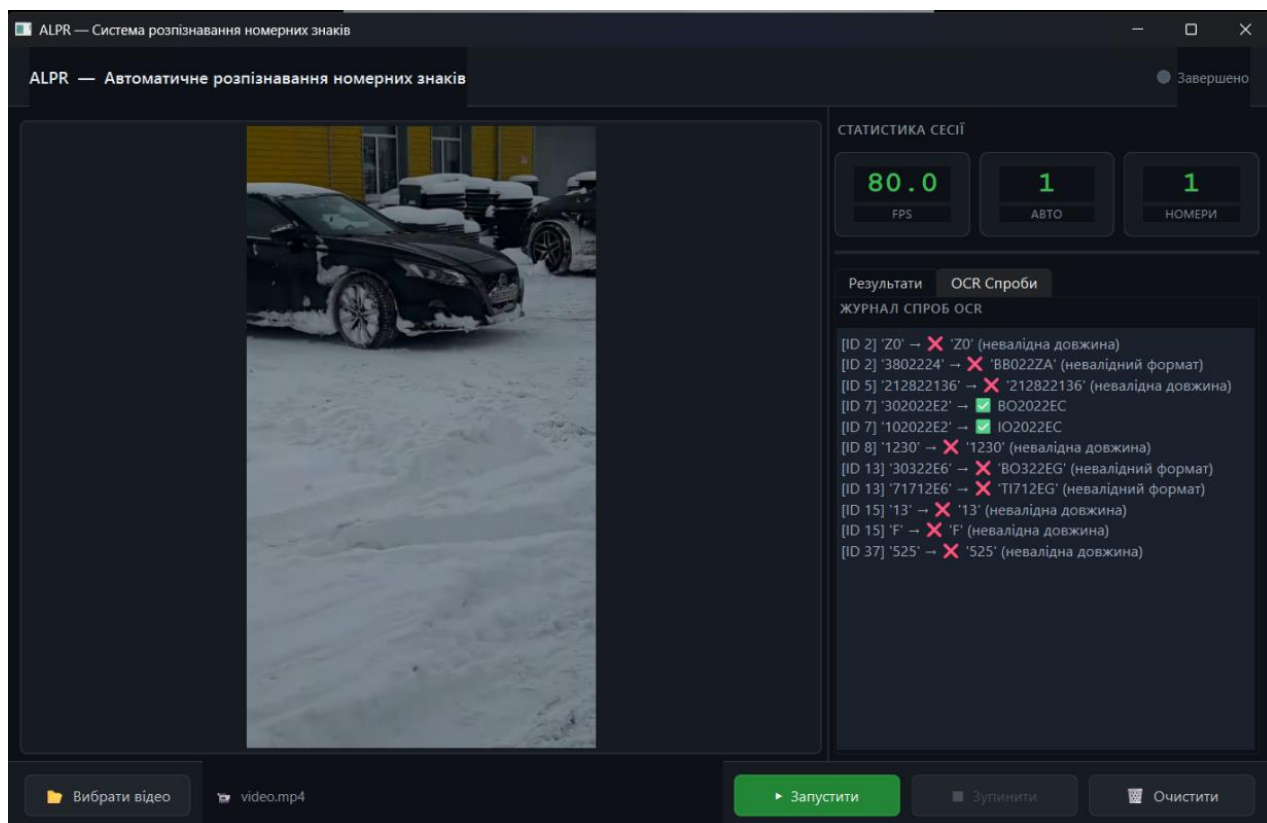


Рисунок 3.3 Процес ітераційного розпізнавання номера в складних умовах.

Аналіз логів на рисунку 3.3 дозволяє зробити такі висновки про реальну роботу системи:

- модуль валідації успішно відсікає проміжні помилки, такі як 302022E2 або 102022E2;
- остаточне рішення приймається лише після накопичення стабільної послідовності, що відповідає ДСТУ (номер BO2022EC);
- система ігнорує візуальний шум (сніг), який нейромережа спочатку сприймає за символи (спроби розпізнавання Z0, 5, F).

Для перевірки роботи з декількома об'єктами одночасно було проведено тест на статичній групі автомобілів (рис. 3.4).

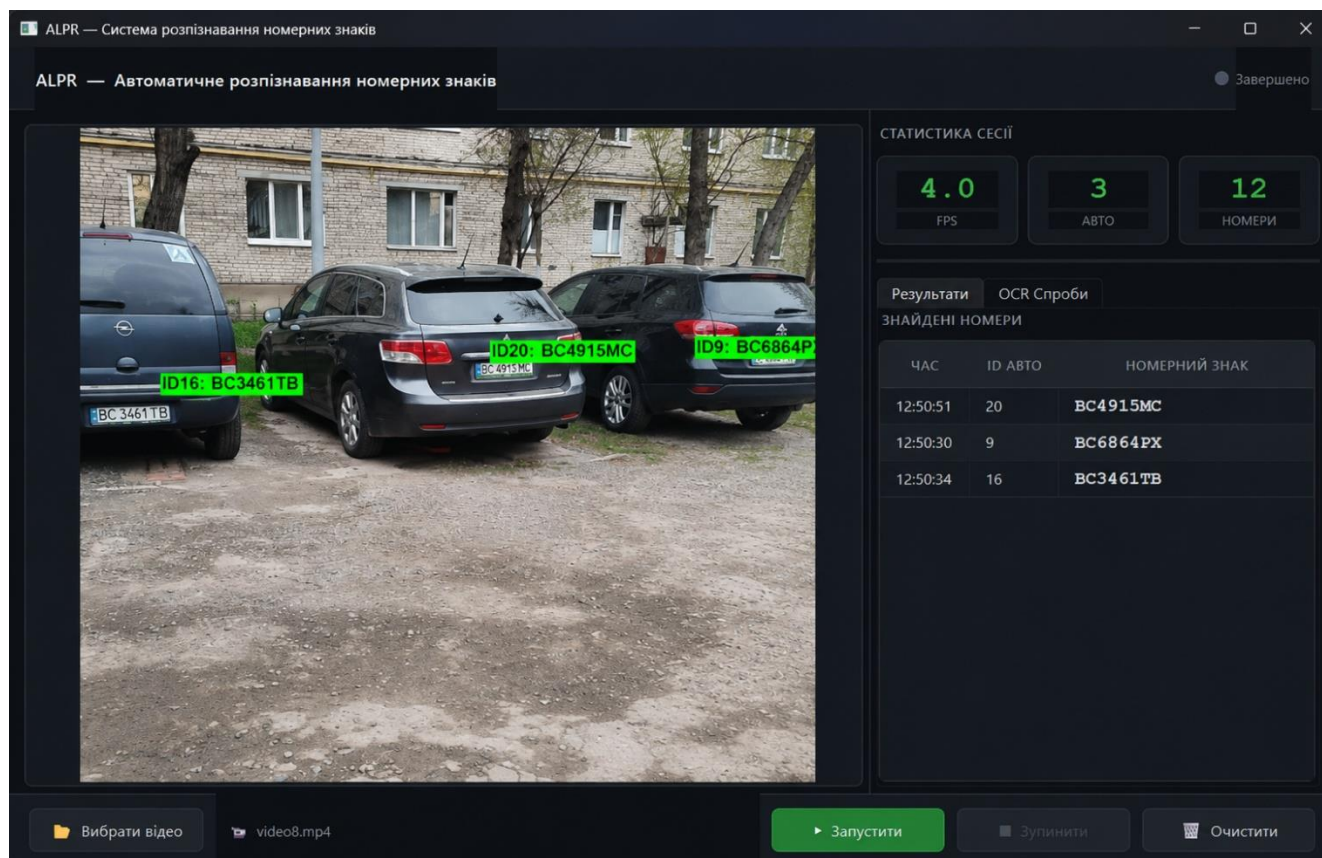


Рисунок 3.4 Одночасна детекція та трекінг декількох об'єктів.

Як видно з рисунка 3.4, система успішно присвоює унікальні ідентифікатори (ID16, ID20) кожному об'єкту та веде паралельне накопичення результатів у таблиці знайдених номерів. Це підтверджує коректність реалізації багатопотокової архітектури та модуля трекінгу.

У процесі тестування було виявлено, що використання лише нейромереж без логічного контролю призводить до значної кількості помилок. Однак справжню ефективність етапу інтелектуальної фільтрації можна відстежити у журналі спроб OCR (див. рис. 3.2). Проаналізувавши наявні логування, бачимо, що система відсікає помилкові спроби за такими критеріями:

- невалідна довжина: видалення послідовностей, що не відповідають стандарту за кількістю знаків;
- невалідний формат: фільтрація рядків, які не проходять перевірку за регулярними виразами (невідповідність структури літерних та цифрових блоків).

3.3.2 Дослідження швидкодії та продуктивності системи

Для оцінки програми важливим критерієм є швидкість розпізнавання кадрів. Під час тестування встановлено, що показник частоти кадрів FPS не є сталою величиною. Визначено, що продуктивність значним чином варіюється залежно від параметрів вхідного відео та навантаження на логічний модуль.

На основі проведених запусків (рис. 3.1–3.4) зафіксовано такі показники швидкодії:

- максимальна продуктивність (80.0 FPS) є зафіксована на відеофрагменті зі складними погодними умовами (рис. 3.3). Такий високий показник свідчить про те, що при оптимізованому розмірі вхідного кадру алгоритми детекції та OCR працюють із значним запасом швидкодії, що перевищує потреби реального часу.
- мінімальна стабільна продуктивність (3.0–5.0 FPS) спостерігається при обробці відео з високою деталізацією та великою кількістю об'єктів у кадрі (рис. 3.1, 3.4). Зниження FPS у цих сценаріях зумовлене високою вартістю обчислювальних операцій при розпізнаванні символів для кожного знайденого об'єкта окремо.

Попри це, навіть при мінімальних показниках, час реакції системи все ще залишається прийнятним для користувача. Завдяки використанню параметра `history_window = 25` система встигає виконати повний цикл перевірки номерного знака приблизно за 5 секунд перебування автомобіля в зоні видимості камери. Це підтверджує, що навіть за умов пікового навантаження архітектура програми зберігає стабільність і ефективність роботи.

3.3.3. Порівняння з існуючими аналогами

Для об'єктивної оцінки розробленої системи проведено порівняння її характеристик із аналогами (табл.3.1), розглянутими у підрозділі 1.2.

Таблиця 3.1 - Порівняння розробленої системи з аналогами

Характеристика	Platerecognizer	OpenALPR	OpenCV+Tesseract	Розроблена система
Локальна робота	Ні	Так	Так	Так
Трекінг кількох об'єктів	Так	Ні	Ні	Так
Підтримка UA стандарту	Так	Частково	Ні	Так
Точність у складних умовах	Висока	Середня	Низька	Середня-висока
Стабілізація результатів	Не описано	Ні	Ні	Так
Графічний інтерфейс	Вебінтерфейс	Ні	Ні	Так
Безкоштовне використання	Обмежено	Так	Так	Так
Відкритий код	Ні	Так	Так	Так

Розроблена система поступається Platerecognizer за точністю розпізнавання — комерційний сервіс навчений на значно більших датасетах і оптимізований для промислового застосування. Також Platerecognizer підтримує трекінг на рівні API, що є перевагою для хмарних інтеграцій.

Порівняно з OpenALPR і підходом OpenCV+Tesseract розроблена система демонструє кращі результати завдяки використанню EasyOCR (архітектура CNN+LSTM) замість класичного OCR, а також завдяки модулю Rolling Vote, який стабілізує результати між кадрами. Відсутність трекінгу в обох аналогах є суттєвим обмеженням при роботі з відеопотоком.

Таким чином, розроблена система займає проміжну позицію: поступається комерційному рішенню за точністю, але перевищує відкриті аналоги за функціональністю і є повністю безкоштовною та локальною. Це відповідає цілям

курсної роботи — дослідженню гібридного підходу, що поєднує класичні методи комп'ютерного зору з нейромережевим розпізнаванням.

3.4 Тестування модулів програмного комплексу

Було розроблено комплекс модульних тестів (Unit Tests) на базі фреймворку pytest [11], щоб гарантувати стабільність та надійність розробленої системи. Процес тестування був спрямований на автоматичну верифікацію правильності обчислень, обробку граничних значень та контроль цілісності даних при взаємодії між модулями.

Написано 38 різних тестів, які включають у себе різні сценарії, охоплюючи ключові компоненти системи:

- тестування детектора. Тут перевірено коректність повернених типів даних та роботу алгоритмів фільтрації за площею. Підтверджено, що система ігнорує хибні контури, площа яких менша за встановлені дані у конфігурації, а також достовірно розраховує центроїди об'єктів навіть для нульових контурів.
- тестування трекара. Тут перевірено, що логіка присвоєння унікальних ідентифікаторів та їхнє збереження у моменти руху є правильною.
- тестування валідатора. Було проведено повну перевірку логіки перетворення розпізнаних символів у валідні номерні знаки:
 1. очищення тексту від спецсимволів та пробілів;
 2. перевірку допустимої довжини рядка;
 3. інтелектуальну позиційну заміну символів-омогліфів (наприклад, заміна цифри «0» на літеру «O» у літерних серіях префіксів та навпаки у цифрових блоках).

Усі тести пройшли успішно, що є підтвердженням належної реалізації функціональних можливостей системи згідно з технічним завданням.

ВИСНОВКИ

У ході виконання курсової роботи розроблено та досліджено систему автоматичного розпізнавання номерних знаків транспортних засобів. За результатами роботи зроблено такі висновки:

- реалізовано алгоритм локалізації, заснований на математичних перетвореннях: білатеральній фільтрації, детекторі Кенні та апроксимації полігонів. Це забезпечило високу швидкість детекції об'єктів без значних витрат обчислювальних ресурсів.
- побудовано систему трекінгу та валідації на основі моментів зображення. Використання часового вікна (25 кадрів) та регулярних виразів дозволило досягти стабільного розпізнавання в умовах перспективних спотворень і візуальних шумів.
- розроблено багатопотокову архітектуру на базі PyQt6, що забезпечило розділення логіки обробки відеоданих та графічного інтерфейсу. Система зберігає високу чуйність навіть при інтенсивних обчислювальних навантаженнях.
- експериментально підтверджено працездатність алгоритмів у складних умовах (зйомка під кутом, опади). Показники швидкодії (FPS від 5.0 до 80.0) довели адаптивність системи до різної якості вхідного потоку.
- проведено верифікацію програмного комплексу за допомогою 38 модульних тестів. Успішне проходження всіх сценаріїв підтвердило коректність математичних моделей та стабільність програмного коду.

Подальший розвиток системи може передбачати інтеграцію нейромережових детекторів (YOLOv8) для підвищення точності у динамічних сценах, розширення бази регіональних кодів та реалізацію експорту даних до зовнішніх СУБД.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. ДСТУ 4278:2019. Дорожній транспорт. Знаки номерні транспортних засобів. Загальні вимоги. — [Чинний від 2020-01-01]. — Київ : ДП «УкрНДНЦ», 2019. — 28 с.
2. Bradski G. Learning OpenCV 4: Computer Vision with the OpenCV Library / G. Bradski, A. Kaehler. — O'Reilly Media, 2019. — 1012 p.
3. Goodfellow I. Deep Learning / I. Goodfellow, Y. Bengio, A. Courville. — MIT Press, 2016. — 800 p.
4. Szeliski R. Computer Vision: Algorithms and Applications. — 2nd ed. — Springer, 2022. — 945 p.
5. EasyOCR Documentation [Електронний ресурс] / Jaided AI. — 2024. — Режим доступу: <https://www.jaided.ai/easyocr/> (цит. 14.05.2026).
6. NumPy documentation [Електронний ресурс]. — 2024. — Режим доступу: <https://numpy.org/doc/> (цит. 14.05.2026).
7. OpenALPR. Automatic License Plate Recognition Library [Електронний ресурс]. — 2018. — Режим доступу: <https://github.com/openalpr/openalpr> (цит. 14.05.2026).
8. OpenCV 4.x Documentation [Електронний ресурс]. — 2024. — Режим доступу: https://docs.opencv.org/4.x/d0/de1/group_core.html (цит. 14.05.2026).
9. Platerecognizer. Automatic License Plate Recognition API [Електронний ресурс] / Parkrow Inc. — 2024. — Режим доступу: <https://platerecognizer.com/> (цит. 14.05.2026).
10. PyQt6 Reference Guide [Електронний ресурс] / Riverbank Computing. — 2024. — Режим доступу: <https://www.riverbankcomputing.com/static/Docs/PyQt6/> (цит. 14.05.2026).
11. pytest : helps you write better programs [Електронний ресурс]. — 2025. — Режим доступу: <https://docs.pytest.org/> (цит. 14.05.2026).

12. Python Software Foundation. Python Language Reference, version 3.13 [Электронный ресурс]. — 2026. — Режим доступа: <https://docs.python.org/3/> (цит. 14.05.2026).
13. Tesseract OCR Documentation [Электронный ресурс] / Google. — 2024. — Режим доступа: <https://tesseract-ocr.github.io/> (цит. 14.05.2026).